## Assignment 4

Deadline: Sunday April 12, 2026 at 23:59
Type: Individual Assignment
Weight: 5%

**Marking Scheme:**

- Program correctness (75%)
- Program clarity (output format, comments, completeness, readability) (10%)
- Test cases to be comprehensive enough to cover your program to test if it is bug free (15%)

---

## Q1. (35 marks) / File Processing

Assume that the information of all TAs in the ECE department is stored in a file called TAs.txt. To be eligible for a TA position, a student must be currently registered and cannot be an alumnus/alumna. However, the file is corrupted and contains some records of TAs who have already graduated and are therefore not eligible to work as TAs. The format of each TA record in the file is as follows:

*Student_Id First_Name Last_Name Hire_Year Classification(Grad or Alum) number_of_working_hours*

The first line of the file indicates the total number of student records (the maximum number of records is 100).

a) Write a program that reads the file, removes the records of invalid TAs (those whose classification is listed as Alum), and updates the original file.**Here is an example:**

TAs.txt

5

*19577 sarah lee 2018 Grad 20*

*40087 peter johnson 2018 Grad 35*

*22136 rayan lee 2017 Alum 40*

*40143 ahmed mobarak 2019 Grad 36*

*40221 rafel Niro 2018 Alum 24*

---

TAs.txt After Correction:

3

*19577 sarah lee 2018 Grad 20*

*40087 peter johnson 2018 Grad 35*

*40143 ahmed mobarak 2019 Grad 36*

Hint: You can create an array of TA objects when you open the original *TAs.txt* file and read the information of TAs.

b) Implement a function AddNewTA() that prompts the user to enter the information for one TA. The user should first enter the Student_ID, followed by the remaining information. The new TA should then be added to the TAs.txt file.

If the user enters a duplicate Student_ID (i.e., one that already exists in the file), the program must reject the input and continue prompting until a unique ID is provided.

Please note that your program should handle exceptions if the user attempts to enter data of an unexpected type.

**Q2. (40 marks)/ Templates, Operator Overloading, and Inheritance**

In this problem, you will implement a template-based Rank-One (1-dimensional) Tensor class. The tensor should be designed so that its elements can be of a generic type T, rather than being restricted to a specific data type such as double.

With this design, your implementation should support tensors of different data types, for example:

- `RankOneTensorType<double>`
- `RankOneTensorType<string>`
- `RankOneTensorType<CustomClass>`

You are given the following base class:

```
class BaseTensor {
public:
virtual void loadData() = 0;
virtual ~BaseTensor() {}
};
```

A sample declaration of template-based Rank-One Tensor classis provided below:

```
template <typename T>
class RankOneTensorType : public BaseTensor {
private:
    std::vector<T> data;

public:
    RankOneTensorType() : data(0) {}
```

```
    RankOneTensorType(int size) : data(size) {}  // data = vector<T>(size);
    ~RankOneTensorType() = default;

    // Other member functions
};
```

**Function Overriding with Templates**

The function void loadData() is overridden from the base class BaseTensor.

Since tensor initialization depends on the data type, the overridden function should be implemented using template specialization. The examples below illustrate this for double and string.

**Example specializations:**

```
template <>
void RankOneTensorType<double>::loadData() {
    std::fill(data.begin(), data.end(), valueGen())  //valueGen(): Static value generator
}


template <>
void RankOneTensorType<string>::loadData() {
    std::fill(data.begin(), data.end(), std::to_string(valueGen()));
}
```

**Task 2.1 – Operator Overloading**

Implement the following **operator overloads** for the template class RankOneTensorType<T>.

| Operator | Description |
|---|---|
| ++ | Increment each element by 1 |
| + | Add two tensors of the same dimension element wise; invalid_argument should be thrown if two tensor objects' dimensions does not match |
| + | Add one tensor with a scalar value element wise |
| = | Assign one tensor to another of the same dimension; invalid_argument should be thrown if two tensor objects' dimensions does not match |
| >> | Output the tensor's element values |
| << | Input the tensor's element values by console input |
| [ ] | Retrieve value by index; invalid_argument should be thrown if the index is out of the range |

**Task 2.2 – Complete Implementation**

Complete the full implementation of:

```
template <typename T>

class RankOneTensorType
```

**Task 2.3 – Unit Testing**

Create a separate file testDriver.cpp and implement the following test function:

```
void testTensorType() {
RankOneTensorType<string> tensor(2);
tensor[0] = "Hello";
tensor[1] = "Coen244";
cout << tensor;

RankOneTensorType<double> dtensor(2);
dtensor[0] = 0.15;
dtensor[1] = 0.30;
cout << dtensor;

RankOneTensorType<double> dtensor2(2);
dtensor2.loadData();

cout << dtensor2;
cout << dtensor + dtensor2;
cout << dtensor[0];
}
```